

矩阵的舞蹈

多维数组与矩阵模式 · 12题 · C++ & Python 双语对照

NQ055 数组的右上半部分 AcWing 745

给定一个 12×12 的二维数组。计算右上半部分 ($j > i$) 所有元素的平均值或求和。

输入

第一行操作类型(S/M)。第二行0-11的行号。然后144个浮点数。

输出

结果保留1位小数。

范围

-10^6 到 10^6

输入

S
(144 numbers...)

输出

(sum, 1 decimal)

思路 右上三角: $j > i$ 。双层循环 $i:0-11, j:0-11$, 满足 $j > i$ 时累加。注意不包括对角线上元素。S求总和, M求平均除以66。

技巧 右上三角=66个元素。S/M判断用if/else。

C++

```
#include <cstdio>

int main()
{
    char t;
    scanf("%c", &t);
    double a[12][12];

    for (int i = 0; i < 12; i ++ )
        for (int j = 0; j < 12; j ++ )
            scanf("%lf", &a[i][j]);

    int c = 0;
    double s = 0;

    for (int i = 0; i < 12; i ++ )
        for (int j = i + 1; j < 12; j ++ )
        {
            c ++ ;
            s += a[i][j];
        }

    if (t == 'S') printf("%.1lf\n", s);
    else printf("%.1lf\n", s / c);

    return 0;
}
```

Python

```
l=input();t=input().strip();s=0
for i in range(12):
    for j in range(12):
        x=float(input())
        if j>i:s+=x
    print(f'{s if t=="S" else s/66:.1f}')
```

NQ056 数组的左上半部分 AcWing 747

计算12×12矩阵左上半部分($j < 11 - i$)元素的平均值或和。

输入 第一行操作类型(S/M)。然后144个浮点数。

输出 结果保留1位小数。

范围 -10^6 到 10^6

输入

S
(144...)

输出

(sum, 1 decimal)

思路 左上三角： $j < 11 - i$ （反对角线以上）。注意不包括反对角线。66个元素。

技巧 反对角线条件： $i + j < 11$ 。

C++

```
#include <cstdio>

int main()
{
    double a[12][12];
    char t;

    scanf("%c", &t);
    for (int i = 0; i < 12; i ++ )
        for (int j = 0; j < 12; j ++ )
            scanf("%lf", &a[i][j]);

    int c = 0;
    double s = 0;
    for (int i = 0; i < 12; i ++ )
        for (int j = 0; j <= 10 - i; j ++ )
        {
            c ++ ;
            s += a[i][j];
        }

    if (t == 'S') printf("%.1lf\n", s);
    else printf("%.1lf\n", s / c);

    return 0;
}
```

Python

```
t=input().strip();s=0
for i in range(12):
    for j in range(12):
        x=float(input())
        if j<11-i:s+=x
    print(f'{s if t=="S" else s/66:.1f}')
```

NQ057 数组的上方区域 AcWing 749

计算12×12矩阵上方区域(去掉左右两个三角)的元素和或平均值。

输入 操作类型(S/M)+144浮点数。

输出 保留1位小数。

范围 -10^6 到 10^6

输入

S
(144...)

输出

(sum,1 decimal)

思路 上方区域： $i=j$ 和右三角($i+j>=11$)。30个元素。

技巧 两个条件交集：i

C++

```
#include <cstdio>
#include <iostream>

using namespace std;

int main()
{
    char t;
    cin >> t;
    double m[12][12];

    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 0; i < 5; i++)
        for (int j = i + 1; j <= 10 - i; j++)
        {
            c += 1;
            s += m[i][j];
        }
    if (t == 'M') printf("%.1lf\n", s / c);
    else printf("%.1lf\n", s);

    return 0;
}
```

Python

```
t=input().strip();s=c=0
for i in range(12):
    for j in range(12):
        x=float(input())
        if i<j and i+j<11:s+=x;c+=1
    print(f'{s if t=="S" else s/c:.1f}')
```

NQ058 数组的左方区域 AcWing 751

计算 12×12 矩阵左方区域(去掉上下三角)的元素和或平均值。

输入 操作类型(S/M)+144浮点数。

输出 保留1位小数。

范围 -10^6 到 10^6

输入

S
(144...)

输出

(sum, 1 decimal)

思路 左方区域：j=11)。

技巧 左右对称于NQ057。条件交换i和j。

C++

```
#include <iostream>
#include <cstdio>

using namespace std;

int main()
{
    char t;
    cin >> t;

    double m[12][12];
    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 1; i <= 5; i++)
        for (int j = 0; j <= i - 1; j++)
        {
            s += m[i][j];
            c += 1;
        }
    for (int i = 6; i <= 10; i++)
        for (int j = 0; j <= 10 - i; j++)
        {
            s += m[i][j];
            c += 1;
        }
    if (t == 'M') printf("%.1lf\n", s / c);
    else printf("%.1lf\n", s);

    return 0;
}
```

Python

```
t=input().strip();s=c=0
for i in range(12):
    for j in range(12):
        x=float(input())
        if j<i and i+j<11:s+=x;c+=1
    print(f'{s if t=="S" else s/c:.1f}')
```

NQ059 平方矩阵I AcWing 753

输入整数N，输出N×N的回字形矩阵。每个位置的值=min(i+1,j+1,N-i,N-j)。

输入 多个整数N(0结束)。

输出 N×N矩阵，每个数占3字符宽。

范围 1≤N≤100

输入

1
2
3
0

输出

1

1 1
1 1

1 1 1
1 2 1
1 1 1

思路 数学公式法： $val = \min(i, j, n-1-i, n-1-j) + 1$ (0-indexed)。每行空格分隔，每个数占3位。注意N=0结束。

技巧 用数学公式替代模拟填充是理解抽象思维的好题。Python: `f'{x:3d}'` 右对齐。

C++

```
#include <iostream>

using namespace std;

int main()
{
    int n;
    while (cin >> n, n)
    {
        for (int i = 1; i <= n; i++)
        {
            for (int j = 1; j <= n; j++)
            {
                int up = i, down = n - i + 1, left = j, right = n - j + 1;
                cout << min(min(up, down), min(left, right)) << ' ';
            }
            cout << endl;
        }

        cout << endl;
    }

    return 0;
}
```

Python

```
while True:
    n=int(input())
    if n==0:break
    for i in range(n):
        row=[]
        for j in range(n):
            v=min(i,j,n-1-i,n-1-j)+1
            row.append(f'{v:3d}')
        print(''.join(row))
    print()
```

NQ060 数组的右下半部分 AcWing 748

计算 12×12 矩阵右下半部分($j > 10 - i$)的元素和或平均值。

输入 操作类型(S/M)+144浮点数。

输出 保留1位小数。

范围 -10^6 到 10^6

输入

S
(144...)

输出

(sum, 1 decimal)

思路 右下三角： $j > 10 - i$ (即 $i + j > 10$)。66个元素。

技巧 $i + j > 10$ 或 $j \geq 11 - i$ 。注意对角线归属。

C++

```
#include <cstdio>
#include <iostream>

using namespace std;

int main()
{
    char t;
    cin >> t;
    double m[12][12];

    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 1; i <= 11; i++)
        for (int j = 12 - i; j <= 11; j++)
        {
            s += m[i][j];
            c += 1;
        }

    if (t == 'S') printf("%.1lf\n", s);
    else printf("%.1lf\n", s / c);

    return 0;
}
```

Python

```
t=input().strip();s=0
for i in range(12):
    for j in range(12):
        x=float(input())
        if i+j>10:s+=x
    print(f'{s if t=="S" else s/66:.1f}')
```

NQ061 数组的左下半部分 AcWing 746

计算 12×12 矩阵左下半部分(j

输入 操作类型(S/M)+144浮点数。

输出 保留1位小数。

范围 -10^6 到 10^6

输入
S
(144...)

输出
(sum,1 decimal)

思路 左下三角：j

技巧 与右上三角对称。

C++

```

#include <stdio>
#include <iostream>

using namespace std;

int main()
{
    char t;
    cin >> t;

    double m[12][12];
    for (int i = 0; i < 12; i ++ )
        for (int j = 0; j < 12; j ++ )
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 0; i < 12; i ++ )
        for (int j = 0; j <= i - 1; j ++ )
        {
            s += m[i][j];
            c += 1;
        }

    if (t == 'S') printf("%.1lf\n", s);
    else printf("%.1lf\n", s / c);

    return 0;
}

```

Python

```

t=input().strip();s=0
for i in range(12):
    for j in range(12):
        x=float(input())
        if j<i:s+=x
    print(f'{s if t=="S" else s/66:.1f}')

```

NQ062 数组的下方区域 AcWing 750

计算12×12矩阵下方区域的元素和或平均值。

输入 操作类型(S/M)+144浮点数。

输出 保留1位小数。

范围 -10^6 到 10^6

输入

S
(144...)

输出

(sum,1 decimal)

思路 下方区域： $i > j$ 且 $i + j > 10$ 。去掉上方三角和左右区域。30个元素。

技巧 与上方区域对称： $i > j$ and $i + j > 10$ 。

C++

```
#include <iostream>
#include <cstdio>

using namespace std;

int main()
{
    char t;
    cin >> t;

    double m[12][12];
    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 7; i <= 11; i++)
        for (int j = 12 - i; j <= i - 1; j++)
        {
            c += 1;
            s += m[i][j];
        }
    if (t == 'M') printf("%.1lf\n", s / c);
    else printf("%.1lf\n", s);

    return 0;
}
```

Python

```
t=input().strip();s=c=0
for i in range(12):
    for j in range(12):
        x=float(input())
        if i>j and i+j>10:s+=x;c+=1
print(f'{s if t=="S" else s/c:.1f}')
```

NQ063 数组的右方区域 AcWing 752

计算12×12矩阵右方区域的元素和或平均值。

输入 操作类型(S/M)+144浮点数。

输出 保留1位小数。

范围 -10^6 到 10^6

输入

S
(144...)

输出

(sum, 1 decimal)

思路 右方区域： $j > i$ 且 $i+j > 10$ 。30个元素。

技巧 与左方区域对称： $j > i$ and $i+j > 10$ 。

C++

```
#include <iostream>
#include <cstdio>

using namespace std;

int main()
{
    char t;
    cin >> t;
    double m[12][12];

    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++)
            cin >> m[i][j];

    double s = 0, c = 0;
    for (int i = 1; i <= 5; i++)
        for (int j = 12 - i; j <= 11; j++)
        {
            s += m[i][j];
            c += 1;
        }
    for (int i = 6; i <= 10; i++)
        for (int j = i + 1; j <= 11; j++)
        {
            s += m[i][j];
            c += 1;
        }
    if (t == 'M') printf("%.1lf\n", s / c);
    else printf("%.1lf\n", s);

    return 0;
}
```

Python

```
t=input().strip();s=c=0
for i in range(12):
    for j in range(12):
        x=float(input())
        if j>i and i+j>10:s+=x;c+=1
    print(f'{s if t=="S" else s/c:.1f}')
```

NQ064 平方矩阵II

AcWing 754

输入N，输出N×N矩阵。每个位置的值=|i-j|+1。

输入 多个整数N(0结束)。

输出 N×N矩阵，每个数占3字符宽。

范围 1≤N≤100

输入

3
0

输出

```
1 2 3
2 1 2
3 2 1
```

思路 对角线距离公式：abs(i-j)+1。

技巧 |i-j|+1巧妙的数学化简。

C++

```
#include <iostream>

using namespace std;

int q[100][100];

int main()
{
    int n;
    while (cin >> n, n)
    {
        for (int i = 0; i < n; i++)
        {
            q[i][i] = 1;
            for (int j = i + 1, k = 2; j < n; j++, k++) q[i][j] = k;
            for (int j = i + 1, k = 2; j < n; j++, k++) q[j][i] = k;
        }

        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++) cout << q[i][j] << ' ';
            cout << endl;
        }
        cout << endl;
    }

    return 0;
}
```

Python

```
while True:
    n=int(input())
    if n==0:break
    for i in range(n):
        print(''.join(f'{abs(i-j)+1:3d}' for j in range(n)))
    print()
```

NQ065 平方矩阵III AcWing 755

输入N，输出N×N矩阵。每个位置的值=2^(i+j)。

输入

多个整数N(0结束)。

输出

N×N矩阵，每个数右对齐到最大数的宽度+1。

范围

1≤N≤15

输入

3
0

输出

```
1  2  4
2  4  8
4  8 16
```

思路 2^(i+j)模式。注意16×16=256，N≤15确保结果在int范围。用1<<(i+j)位运算。

技巧 C++用1<<(i+j)位运算。Python的2** (i+j)或1<<(i+j)。右对齐宽度=最长数字宽度+1。

C++

```
#include <iostream>
#include <cstdio>

using namespace std;

int main()
{
    int n;
    while (cin >> n, n)
    {
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                int v = 1;
                for (int k = 0; k < i + j; k++) v *= 2;
                cout << v << ' ';
            }
            cout << endl;
        }
        cout << endl;
    }
    return 0;
}
```

Python

```
while True:
    n=int(input())
    if n==0:break
    w=len(str(2**(2*n-2)))+1
    for i in range(n):
        print(''.join(f'{2**(i+j):{w}d}' for j in range(n)))
    print()
```

NQ066 蛇形矩阵 AcWing 756

输入n和m，输出n×m蛇形矩阵。从(1,1)开始，每次向右走到边界，向下走一步，再向左，向上……循环直到所有格填满。

输入 一行两个整数n和m。

输出 n行m列的蛇形矩阵。

范围 $1 \leq n, m \leq 100$

输入	输出
3 3	1 2 3 8 9 4 7 6 5

思路 方向数组+边界检测。d=[(0,1),(1,0),(0,-1),(-1,0)]依次控制→↓←↑。撞墙或已填则转向。while循环填充所有格子。

技巧 方向数组是处理螺旋/蛇形遍历的标准模式。每次移动前检查是否越界或已填。

C++

```

#include <iostream>

using namespace std;

int res[100][100];

int main()
{
    int n, m;
    cin >> n >> m;

    int dx[] = {0, 1, 0, -1}, dy[] = {1, 0, -1, 0};

    for (int x = 0, y = 0, d = 0, k = 1; k <= n * m; k++)
    {
        res[x][y] = k;
        int a = x + dx[d], b = y + dy[d];
        if (a < 0 || a >= n || b < 0 || b >= m || res[a][b])
        {
            d = (d + 1) % 4;
            a = x + dx[d], b = y + dy[d];
        }
        x = a; y = b;
    }

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++) cout << res[i][j] << ' ';
        cout << endl;
    }

    return 0;
}

```

Python

```

n,m=map(int,input().split());a=[[0]*m for _ in range(n)]
dx=[0,1,0,-1];dy=[1,0,-1,0];d=0;x=y=0
for v in range(1,n*m+1):
    a[x][y]=v;nx=x+dx[d];ny=y+dy[d]
    if nx<0 or nx>=n or ny<0 or ny>=m or a[nx][ny]:d=(d+1)%4;nx=x+dx[d];ny=y+dy[d]
    x,y=nx,ny
for r in a:print(' '.join(map(str,r)))

```